Data quality

October 26, 2026

1 Project work, part 2 - Data sources

GitHub Repository: https://github.com/sgkop/IND320-Innlevering-sgkop

Streamlit App: https://ind320-innlevering1-sgkop.streamlit.app/

1.1 Setup

[12]:
```
import pandas as pd
import numpy as np
import requests
import geopandas as gpd
import matplotlib.pyplot as plt
```

[13]:
```
import requests

url = (
    "https://kart.nve.no/enterprise/rest/services/"
    "Mapservices/Administrasjon2/MapServer/0/query"
)

params = {
    "where": "1=1",
    "outFields": "*",
    "returnGeometry": True,
    "f": "json"
}

data = requests.get(
    url,
    params=params
).json()
```

[14]:
```
for field in data["fields"]:
    print(field["name"])
```

```
Shape_Area
Shape_Length
OBJECTID
```

```
REGION
REGIONNAVN
POSTNUMMER
POSTSTED
BESØKSADR
ADRESSE
```

[15]:
```
for feature in data["features"]:
    print(feature["attributes"]["REGION"])
```

```
RV
RM
RN
RØ
RS
```

[16]:
```
regions_df = pd.DataFrame(
    [
        feature["attributes"]
        for feature in data["features"]
    ]
)

regions_df
```

[16]:

	Shape_Area	Shape_Length	OBJECTID	REGION	REGIONNAVN	POSTNUMMER	\
0	7.541396e+10	1.673059e+06	1	RV	Region Vest	6801	
1	8.874660e+10	1.952347e+06	2	RM	Region Midt-Norge	7030	
2	1.675807e+11	2.886733e+06	3	RN	Region Nord	8514	
3	6.368011e+10	1.595783e+06	4	RØ	Region Øst	2307	
4	7.468574e+10	1.405013e+06	5	RS	Region Sør	3103	

	POSTSTED	BESØKSADR	ADRESSE
0	Førde	Naustdalsveien 1B	Postboks 53
1	Trondheim	Abelsgate 9, Trondheim	Abelsgate 9
2	Narvik	Kongensgate 52-54	Kongensgate 52-54
3	Hamar	Vangsvegen 73	Postboks 4223
4	Tønsberg	Anton Jenssens gate 7	Postboks 2124

[17]:
```
# Region overview
regions_df[
    ["REGION", "REGIONNAVN"]
]
```

[17]:

	REGION	REGIONNAVN
0	RV	Region Vest
1	RM	Region Midt-Norge
2	RN	Region Nord

```
3     RØ          Region Øst
4     RS          Region Sør
```

[18]:
```
data["features"][0].keys()
```

[18]:
```
dict_keys(['attributes', 'geometry'])
```

[19]:
```
data["features"][0]["geometry"].keys()
```

[19]:
```
dict_keys(['rings'])
```

[20]:
```
len(data["features"][0]["geometry"]["rings"][0])
```

[20]:
```
3123
```

[21]:
```
# Create GeoDataFrame
from shapely.geometry import Polygon
import geopandas as gpd

records = []

for feature in data["features"]:

    attrs = feature["attributes"]

    polygon = Polygon(
        feature["geometry"]["rings"][0]
    )

    attrs["geometry"] = polygon

    records.append(attrs)

regions_gdf = gpd.GeoDataFrame(
    records,
    geometry="geometry",
    crs="EPSG:25833"
)

regions_gdf
```

[21]:

	Shape_Area	Shape_Length	OBJECTID	REGION	REGIONNAVN	POSTNUMMER	\
0	7.541396e+10	1.673059e+06	1	RV	Region Vest	6801	
1	8.874660e+10	1.952347e+06	2	RM	Region Midt-Norge	7030	
2	1.675807e+11	2.886733e+06	3	RN	Region Nord	8514	
3	6.368011e+10	1.595783e+06	4	RØ	Region Øst	2307	
4	7.468574e+10	1.405013e+06	5	RS	Region Sør	3103	

```
    POSTSTED               BESØKSADR             ADRESSE  \
0      Førde       Naustdalsveien 1B         Postboks 53
1  Trondheim  Abelsgate 9, Trondheim         Abelsgate 9
2     Narvik       Kongensgate 52-54  Kongensgate 52-54
3      Hamar           Vangsvegen 73       Postboks 4223
4   Tønsberg   Anton Jenssens gate 7       Postboks 2124

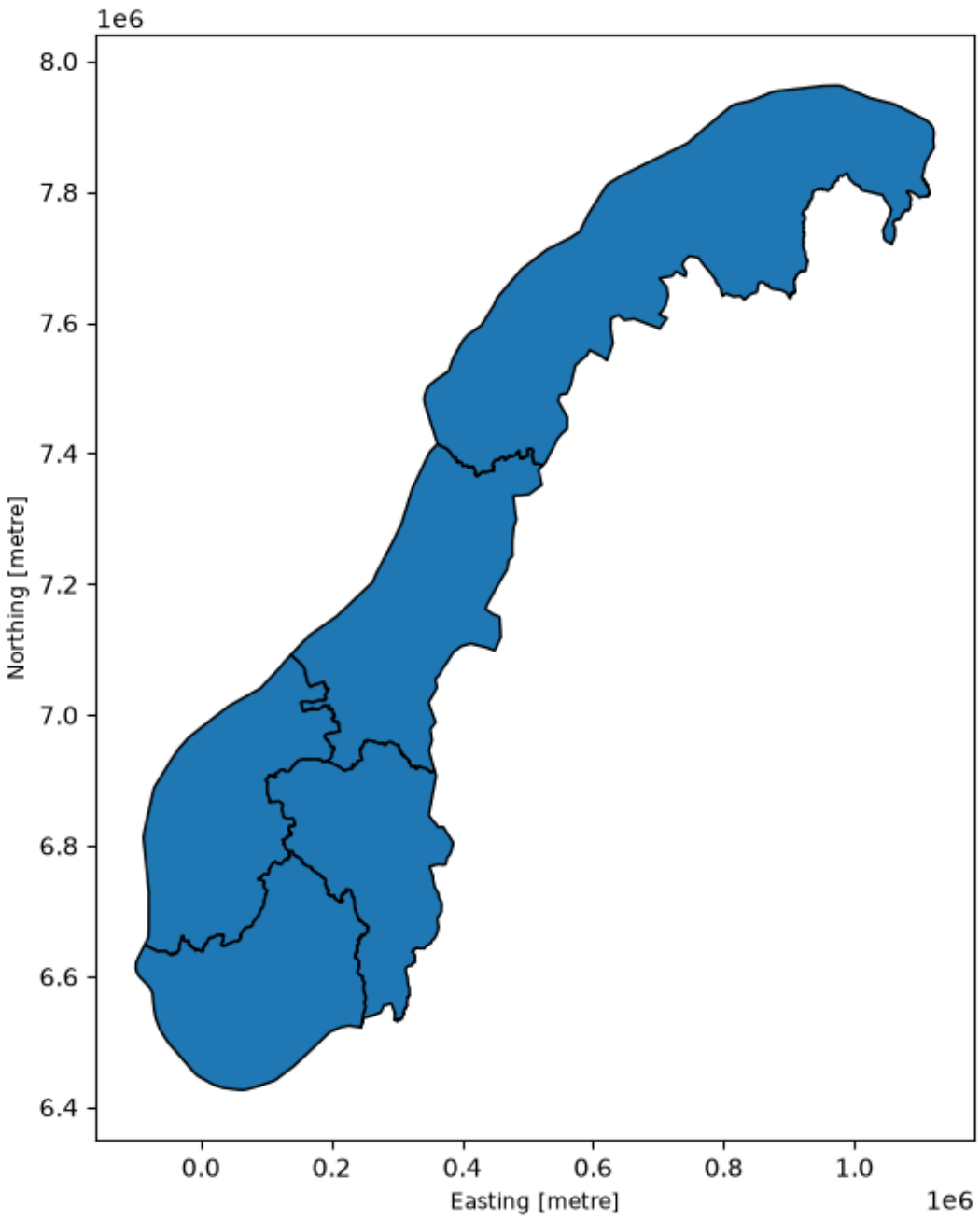
                                              geometry
0  POLYGON ((193105.76 6929797.839, 191705.75 692…
1  POLYGON ((391250 7388620, 392270 7386790, 3937…
2  POLYGON ((1109096.413 7844647.502, 1105115.343…
3  POLYGON ((253748.22 6678130.91, 253645.58 6678…
4  POLYGON ((253748.22 6678130.91, 253917.63 6677…
```

[22]:
```
regions_gdf[
    ["REGION", "REGIONNAVN"]
]
```

[22]:
```
  REGION         REGIONNAVN
0     RV        Region Vest
1     RM  Region Midt-Norge
2     RN        Region Nord
3     RØ         Region Øst
4     RS         Region Sør
```

[23]:
```
regions_gdf.plot(
    figsize=(8, 8),
    edgecolor="black"
)
```

[23]:
```
<Axes: xlabel='Easting [metre]', ylabel='Northing [metre]'>
```



[24]:
```
# Random Sampling
import random
from shapely.geometry import Point

def sample_points(
    polygon,
    n=1000,
```

```
    seed=42
):

    random.seed(seed)

    minx, miny, maxx, maxy = polygon.bounds

    points = []

    while len(points) < n:

        point = Point(
            random.uniform(minx, maxx),
            random.uniform(miny, maxy)
        )

        if polygon.contains(point):
            points.append(point)

    return points
```

[25]:
```
rm_polygon = regions_gdf.loc[
    regions_gdf["REGION"] == "RM",
    "geometry"
].iloc[0]

rm_points = sample_points(
    rm_polygon,
    n=1000
)

len(rm_points)
```

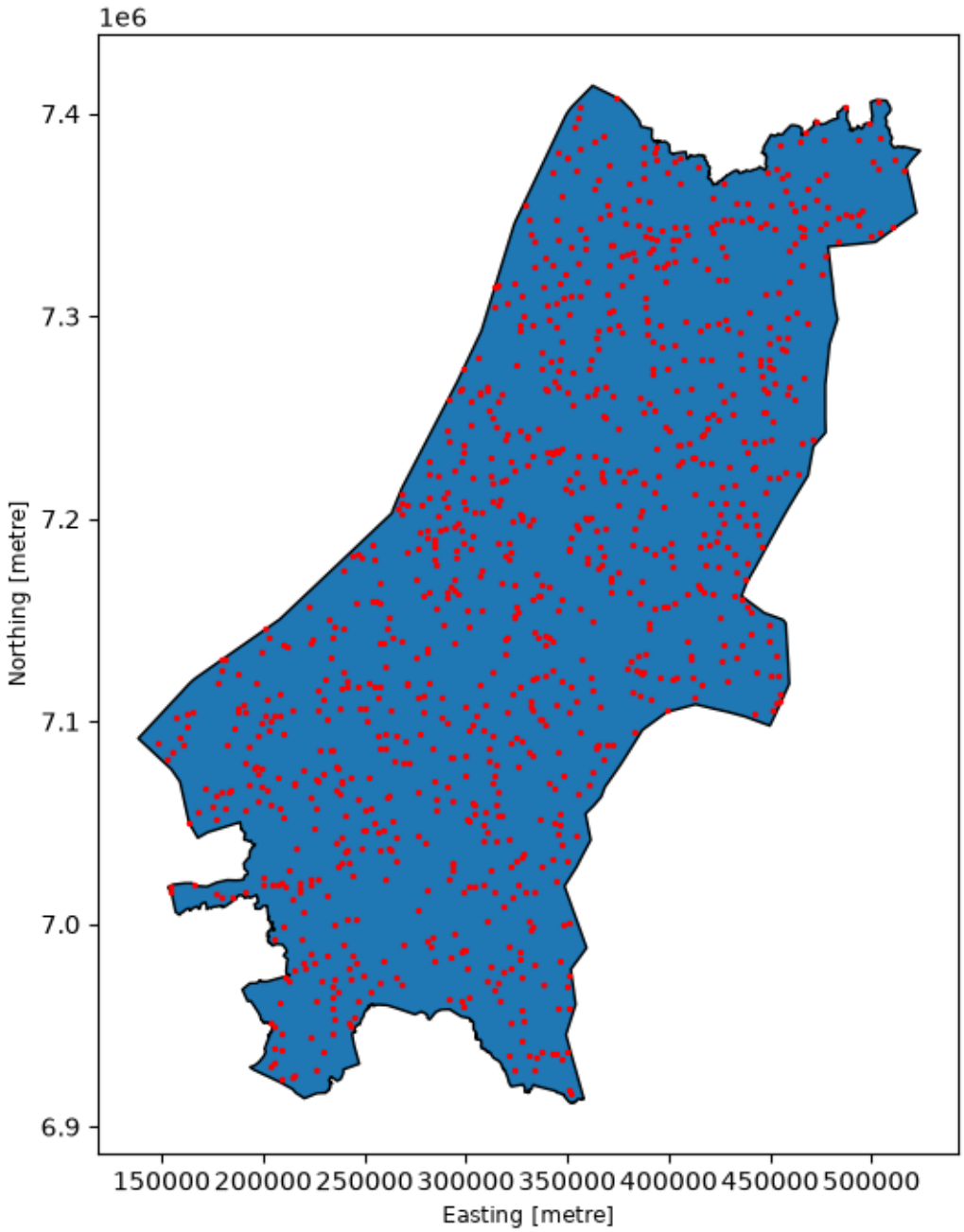
[25]: 1000

[26]:
```
points_gdf = gpd.GeoDataFrame(
    geometry=rm_points,
    crs=regions_gdf.crs
)

ax = regions_gdf[
    regions_gdf["REGION"] == "RM"
].plot(
    figsize=(8,8),
    edgecolor="black"
)
```

```
points_gdf.plot(
    ax=ax,
    color="red",
    markersize=2
)
```

[26]: <Axes: xlabel='Easting [metre]', ylabel='Northing [metre]'>



[27]:
```
# Coordinate Conversion
points_gdf.head()
```

[27]:
```
                        geometry
0   POINT (244096.902 7024100.86)
1  POINT (422049.159 7251884.554)
2  POINT (388657.496 7185703.584)
3  POINT (448792.807 7262653.644)
4   POINT (269249.06 6990080.012)
```

[28]:
```
points_wgs84 = points_gdf.to_crs(
    "EPSG:4326"
)

points_wgs84.head()
```

[28]:
```
                     geometry
0   POINT (9.89948 63.25427)
1  POINT (13.32294 65.38037)
2  POINT (12.65794 64.77738)
3  POINT (13.89408 65.48227)
4  POINT (10.44677 62.96695)
```

[29]:
```
points_wgs84["longitude"] = (
    points_wgs84.geometry.x
)

points_wgs84["latitude"] = (
    points_wgs84.geometry.y
)

points_wgs84[
    ["latitude", "longitude"]
].head()
```

[29]:
```
    latitude  longitude
0  63.254269   9.899484
1  65.380372  13.322943
2  64.777381  12.657935
3  65.482267  13.894083
4  62.966950  10.446766
```

[30]:
```
sample_point = points_wgs84.iloc[0]

sample_point[
    ["latitude", "longitude"]
]
```

[30]:
```
latitude     63.254269
longitude     9.899484
Name: 0, dtype: object
```

[31]:
```
# Open-Meteo test
lat = sample_point["latitude"]
lon = sample_point["longitude"]

print(lat, lon)
```

```
63.25426937785838 9.899483754425445
```

[32]:
```
url = (
    "https://archive-api.open-meteo.com/v1/archive"
)

params = {
    "latitude": lat,
    "longitude": lon,
    "start_date": "2024-01-01",
    "end_date": "2024-01-07",
    "hourly": [
        "temperature_2m",
        "precipitation"
    ]
}

response = requests.get(
    url,
    params=params
)

weather_data = response.json()

weather_data.keys()
```

[32]:
```
dict_keys(['latitude', 'longitude', 'generationtime_ms', 'utc_offset_seconds',
'timezone', 'timezone_abbreviation', 'elevation', 'hourly_units', 'hourly'])
```

[33]:
```
weather_data["hourly"].keys()
```

[33]:
```
dict_keys(['time', 'temperature_2m', 'precipitation'])
```

[34]:
```
weather_df = pd.DataFrame(
    weather_data["hourly"]
)

weather_df.head()
```

[34]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-6.4	0.0
1	2024-01-01T01:00	-6.3	0.0
2	2024-01-01T02:00	-6.4	0.0
3	2024-01-01T03:00	-6.4	0.0
4	2024-01-01T04:00	-6.4	0.0

[35]:
```
# Weather Download Function
def download_weather(
    latitude,
    longitude,
    start_date,
    end_date
):

    url = (
        "https://archive-api.open-meteo.com/v1/archive"
    )

    params = {
        "latitude": latitude,
        "longitude": longitude,
        "start_date": start_date,
        "end_date": end_date,
        "hourly": [
            "temperature_2m",
            "precipitation"
        ]
    }

    response = requests.get(
        url,
        params=params
    )

    data = response.json()

    return pd.DataFrame(
        data["hourly"]
    )
```

[36]:
```
weather_df = download_weather(
    latitude=sample_point["latitude"],
    longitude=sample_point["longitude"],
    start_date="2024-01-01",
    end_date="2024-01-07"
)
```

```
weather_df.head()
```

[36]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-6.4	0.0
1	2024-01-01T01:00	-6.3	0.0
2	2024-01-01T02:00	-6.4	0.0
3	2024-01-01T03:00	-6.4	0.0
4	2024-01-01T04:00	-6.4	0.0

[37]:
```
def get_region_points(
    region_code,
    n=1000
):

    polygon = regions_gdf.loc[
        regions_gdf["REGION"] == region_code,
        "geometry"
    ].iloc[0]

    points = sample_points(
        polygon,
        n=n
    )

    points_gdf = gpd.GeoDataFrame(
        geometry=points,
        crs=regions_gdf.crs
    )

    points_wgs84 = points_gdf.to_crs(
        "EPSG:4326"
    )

    points_wgs84["longitude"] = (
        points_wgs84.geometry.x
    )

    points_wgs84["latitude"] = (
        points_wgs84.geometry.y
    )

    return points_wgs84[
        ["latitude", "longitude"]
    ]
```

[38]:
```
rm_points = get_region_points("RM")

rm_points.head()
```

[38]:

	latitude	longitude
0	63.254269	9.899484
1	65.380372	13.322943
2	64.777381	12.657935
3	65.482267	13.894083
4	62.966950	10.446766

[39]:
```
# Weather Download Function
def download_weather(
    latitude,
    longitude,
    start_date,
    end_date
):

    url = (
        "https://archive-api.open-meteo.com/v1/archive"
    )

    params = {
        "latitude": latitude,
        "longitude": longitude,
        "start_date": start_date,
        "end_date": end_date,
        "hourly": [
            "temperature_2m",
            "precipitation"
        ]
    }

    response = requests.get(
        url,
        params=params
    )

    data = response.json()

    return pd.DataFrame(
        data["hourly"]
    )
```

[40]:
```
weather_df = download_weather(
    latitude=rm_points.iloc[0]["latitude"],
```

```
    longitude=rm_points.iloc[0]["longitude"],
    start_date="2024-01-01",
    end_date="2024-01-07"
)

weather_df.head()
```

[40]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-6.4	0.0
1	2024-01-01T01:00	-6.3	0.0
2	2024-01-01T02:00	-6.4	0.0
3	2024-01-01T03:00	-6.4	0.0
4	2024-01-01T04:00	-6.4	0.0

[41]:
```
def region_weather(
    region_code,
    start_date,
    end_date,
    n=1000
):

    points = get_region_points(
        region_code,
        n=n
    )

    sample_point = points.iloc[0]

    weather_df = download_weather(
        latitude=sample_point["latitude"],
        longitude=sample_point["longitude"],
        start_date=start_date,
        end_date=end_date
    )

    return weather_df
```

[42]:
```
rm_weather = region_weather(
    region_code="RM",
    start_date="2024-01-01",
    end_date="2024-01-07"
)

rm_weather.head()
```

[42]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-6.4	0.0

```
1  2024-01-01T01:00            -6.3            0.0
2  2024-01-01T02:00            -6.4            0.0
3  2024-01-01T03:00            -6.4            0.0
4  2024-01-01T04:00            -6.4            0.0
```

[43]:
```
# Download Weather Data for One Region
rm_points = get_region_points(
    "RM",
    n=10
)

rm_points.head()
```

[43]:
```
    latitude  longitude
0  63.254269   9.899484
1  65.380372  13.322943
2  64.777381  12.657935
3  65.482267  13.894083
4  62.966950  10.446766
```

[44]:
```
weather_1 = download_weather(
    latitude=rm_points.iloc[0]["latitude"],
    longitude=rm_points.iloc[0]["longitude"],
    start_date="2024-01-01",
    end_date="2024-01-31"
)

weather_2 = download_weather(
    latitude=rm_points.iloc[1]["latitude"],
    longitude=rm_points.iloc[1]["longitude"],
    start_date="2024-01-01",
    end_date="2024-01-31"
)

weather_3 = download_weather(
    latitude=rm_points.iloc[2]["latitude"],
    longitude=rm_points.iloc[2]["longitude"],
    start_date="2024-01-01",
    end_date="2024-01-31"
)
```

[45]:
```
plt.figure(figsize=(10,5))

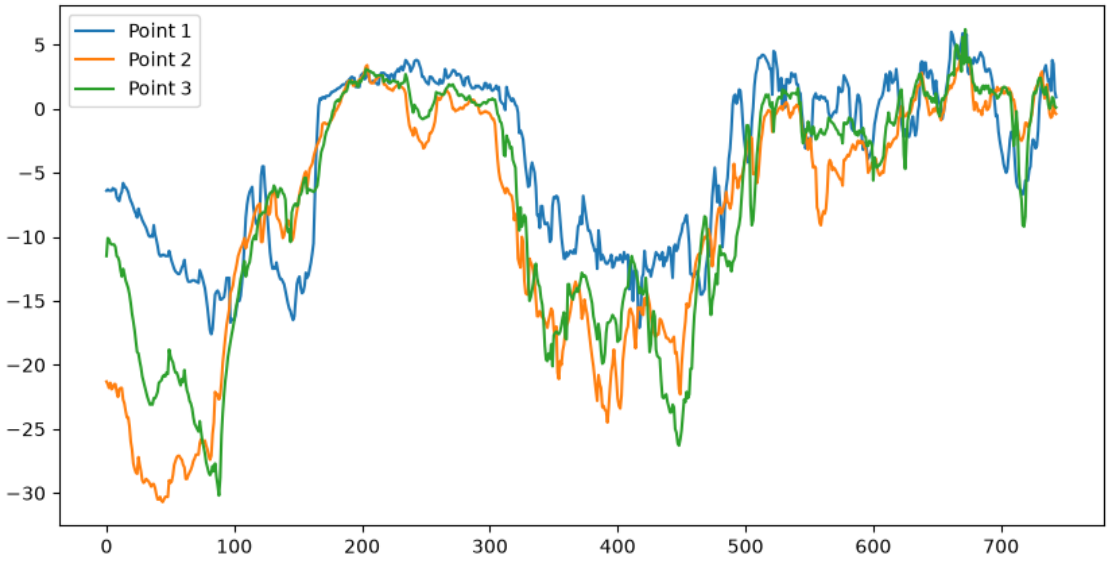
plt.plot(
    weather_1["temperature_2m"],
    label="Point 1"
)
```

```
plt.plot(
    weather_2["temperature_2m"],
    label="Point 2"
)

plt.plot(
    weather_3["temperature_2m"],
    label="Point 3"
)

plt.legend()

plt.show()
```



[46]:
```
# Regional Weather Aggregation
def aggregate_region_weather(
    region_code,
    start_date,
    end_date,
    n_points=10
):

    points = get_region_points(
        region_code,
        n=n_points
    )
```

```
    all_weather = []

    for _, point in points.iterrows():

        weather_df = download_weather(
            latitude=point["latitude"],
            longitude=point["longitude"],
            start_date=start_date,
            end_date=end_date
        )

        all_weather.append(weather_df)

    return all_weather
```

[47]:
```
weather_list = aggregate_region_weather(
    region_code="RM",
    start_date="2024-01-01",
    end_date="2024-01-07",
    n_points=3
)

len(weather_list)
```

[47]:
```
3
```

[48]:
```
weather_list[0].head()
```

[48]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-6.4	0.0
1	2024-01-01T01:00	-6.3	0.0
2	2024-01-01T02:00	-6.4	0.0
3	2024-01-01T03:00	-6.4	0.0
4	2024-01-01T04:00	-6.4	0.0

[49]:
```
regional_weather = weather_list[0].copy()

regional_weather["temperature_2m"] = (
    weather_list[0]["temperature_2m"]
    + weather_list[1]["temperature_2m"]
    + weather_list[2]["temperature_2m"]
) / 3

regional_weather["precipitation"] = (
    weather_list[0]["precipitation"]
    + weather_list[1]["precipitation"]
    + weather_list[2]["precipitation"]
```

```
) / 3

regional_weather.head()
```

[49]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-13.066667	0.0
1	2024-01-01T01:00	-12.633333	0.0
2	2024-01-01T02:00	-12.800000	0.0
3	2024-01-01T03:00	-12.766667	0.0
4	2024-01-01T04:00	-12.966667	0.0

[50]:
```
def mean_weather(weather_list):

    regional_weather = weather_list[0].copy()

    regional_weather["temperature_2m"] = sum(
        df["temperature_2m"]
        for df in weather_list
    ) / len(weather_list)

    regional_weather["precipitation"] = sum(
        df["precipitation"]
        for df in weather_list
    ) / len(weather_list)

    return regional_weather
```

[51]:
```
regional_weather = mean_weather(
    weather_list
)

regional_weather.head()
```

[51]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-13.066667	0.0
1	2024-01-01T01:00	-12.633333	0.0
2	2024-01-01T02:00	-12.800000	0.0
3	2024-01-01T03:00	-12.766667	0.0
4	2024-01-01T04:00	-12.966667	0.0

[52]:
```
rm_weather = mean_weather(
    aggregate_region_weather(
        "RM",
        "2024-01-01",
        "2024-12-31",
        n_points=10
    )
```

```
)
```

[53]:
```
rm_weather.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8784 entries, 0 to 8783
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   time            8784 non-null   str
 1   temperature_2m  8784 non-null   float64
 2   precipitation   8784 non-null   float64
dtypes: float64(2), str(1)
memory usage: 206.0 KB
```

[54]:
```
rm_weather.head()
```

[54]:

	time	temperature_2m	precipitation
0	2024-01-01T00:00	-11.18	0.0
1	2024-01-01T01:00	-10.74	0.0
2	2024-01-01T02:00	-10.89	0.0
3	2024-01-01T03:00	-10.93	0.0
4	2024-01-01T04:00	-10.88	0.0

[55]:
```
# Analysis Settings

REGION_CODE = "RM"
YEAR = 2024

DEV_N_POINTS = 10

DCT_CUTOFF = 20
N_STD = 3
```

[56]:
```
# Regional Weather Analysis
rm_weather["time"] = pd.to_datetime(
    rm_weather["time"]
)
```

[57]:
```
plt.figure(figsize=(12,6))

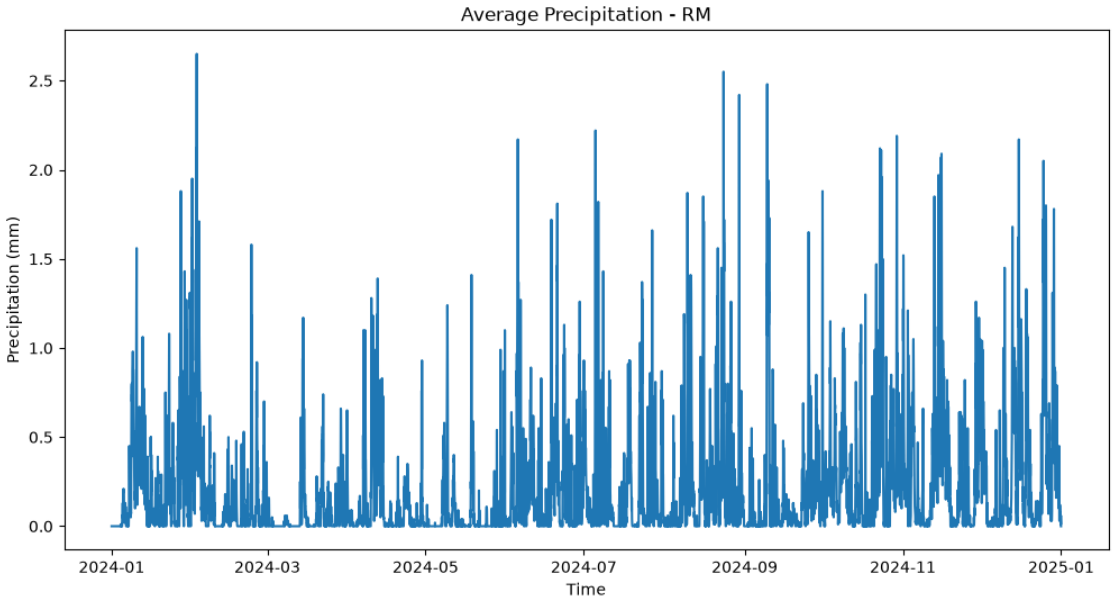
plt.plot(
    rm_weather["time"],
    rm_weather["precipitation"]
)

plt.title(
    "Average Precipitation - RM"
```

```
)

plt.xlabel("Time")
plt.ylabel("Precipitation (mm)")

plt.show()
```



[58]:
```
# Daily Aggregation
daily_weather = (
    rm_weather
    .set_index("time")
    .resample("D")
    .agg({
        "temperature_2m": "mean",
        "precipitation": "sum"
    })
)

daily_weather.head()
```

[58]:

time	temperature_2m	precipitation
2024-01-01	-12.277500	0.00
2024-01-02	-17.686667	0.00
2024-01-03	-18.316250	0.00
2024-01-04	-18.036667	0.08
2024-01-05	-9.338333	2.18

[59]:
```
daily_weather.info()
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 366 entries, 2024-01-01 to 2024-12-31
Freq: D
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   temperature_2m  366 non-null    float64
 1   precipitation   366 non-null    float64
dtypes: float64(2)
memory usage: 8.6 KB
```

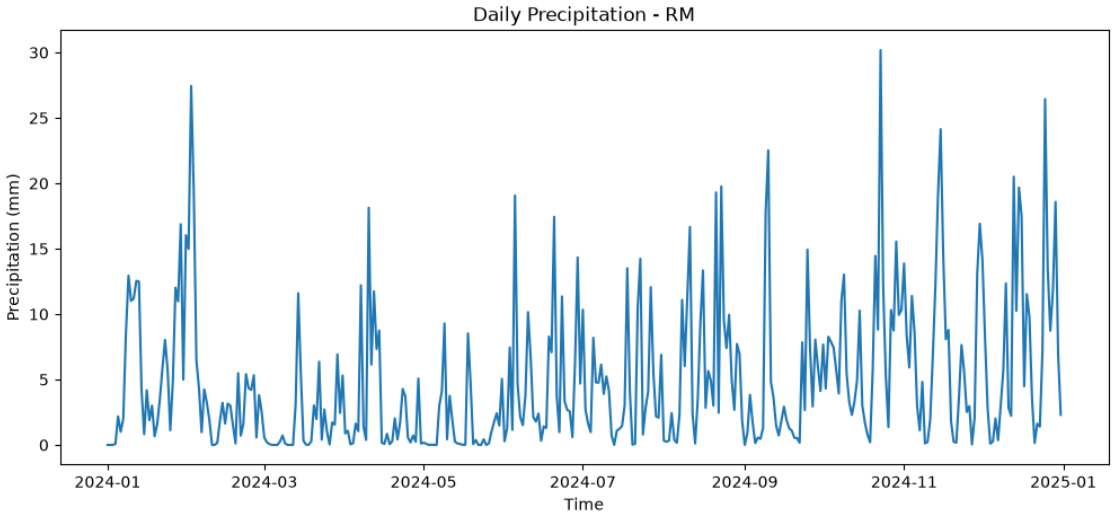
[60]:
```
plt.figure(figsize=(12,5))

plt.plot(
    daily_weather.index,
    daily_weather["precipitation"]
)

plt.title(
    "Daily Precipitation - RM"
)

plt.xlabel("Time")
plt.ylabel("Precipitation (mm)")

plt.show()
```



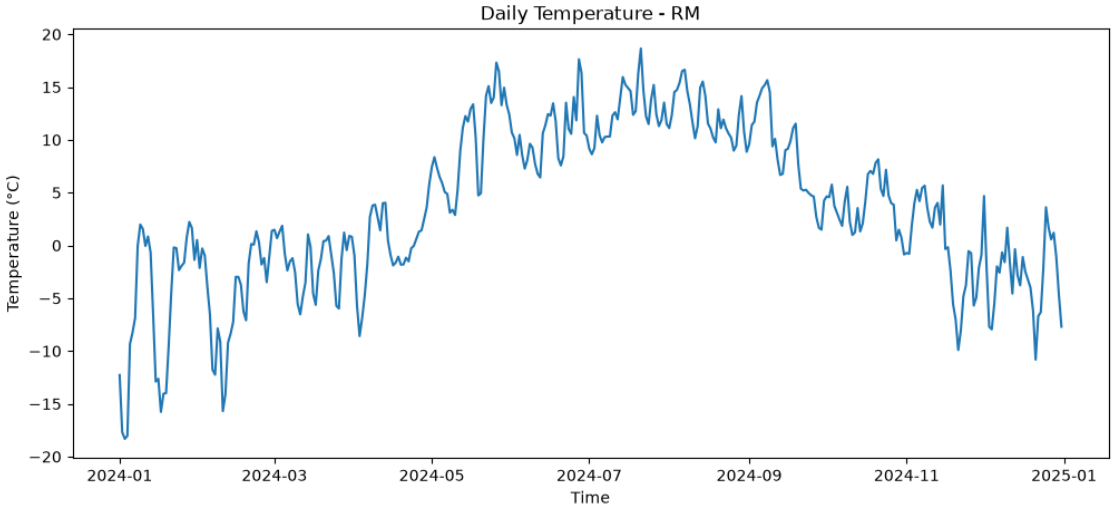
[61]:
```
plt.figure(figsize=(12,5))

plt.plot(
    daily_weather.index,
    daily_weather["temperature_2m"]
)

plt.title(
    "Daily Temperature - RM"
)

plt.xlabel("Time")
plt.ylabel("Temperature (°C)")

plt.show()
```



2 SPC and DCT Analysis

[62]:
```
from scipy.fftpack import dct, idct
```

[63]:
```
def precipitation_spc(
    precipitation_series,
    dct_cutoff=20,
    n_std=3
):

    signal = precipitation_series.values
```

```
    coeffs = dct(
        signal,
        norm="ortho"
    )

    filtered_coeffs = coeffs.copy()

    filtered_coeffs[:dct_cutoff] = 0

    satv = idct(
        filtered_coeffs,
        norm="ortho"
    )

    median = np.median(satv)

    mad = np.median(
        np.abs(satv - median)
    )

    robust_std = 1.4826 * mad

    upper_limit = (
        median
        + n_std * robust_std
    )

    lower_limit = (
        median
        - n_std * robust_std
    )

    outliers = (
        (satv > upper_limit)
        |
        (satv < lower_limit)
    )

    plt.figure(figsize=(12,5))

    plt.plot(
        precipitation_series.index,
        signal,
        label="Precipitation"
    )

    plt.axhline(
```

```
        upper_limit,
        color="red",
        linestyle="--",
        label="Upper SPC limit"
    )

    plt.axhline(
        lower_limit,
        color="red",
        linestyle="--",
        label="Lower SPC limit"
    )

    plt.scatter(
        precipitation_series.index[outliers],
        signal[outliers],
        color="orange",
        label="Outliers"
    )

    plt.legend()

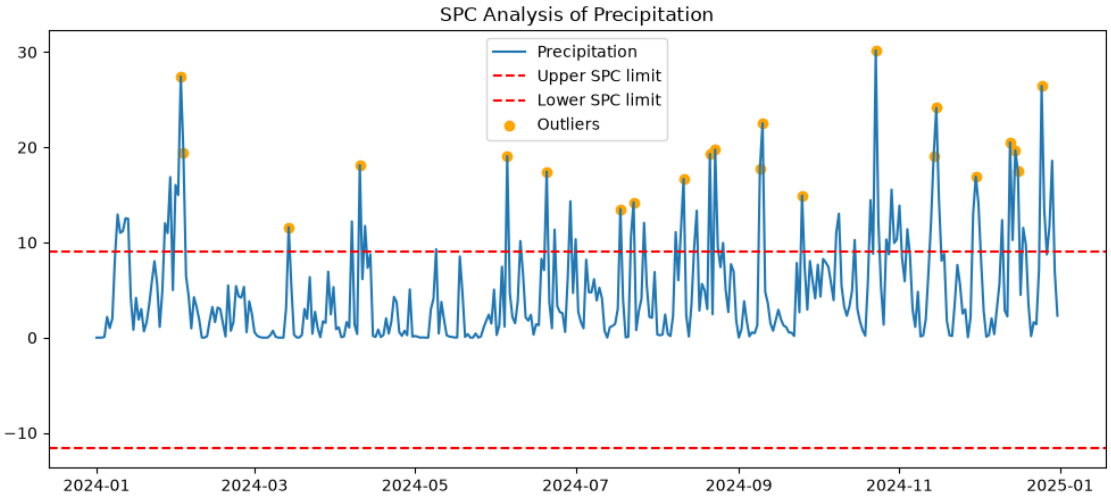
    plt.title(
        "SPC Analysis of Precipitation"
    )

    plt.show()

    return {
        "n_outliers": outliers.sum(),
        "upper_limit": upper_limit,
        "lower_limit": lower_limit,
        "outliers": outliers
    }
```

[64]:
```
spc_results = precipitation_spc(
    daily_weather["precipitation"],
    dct_cutoff=DCT_CUTOFF,
    n_std=N_STD
)

spc_results["n_outliers"]
```



[64]:
```
np.int64(22)
```

[65]:
```
# LOF Analysis
from sklearn.neighbors import LocalOutlierFactor
```

[66]:
```
def temperature_lof(
    temperature_series,
    contamination=0.01
):

    values = temperature_series.values.reshape(-1, 1)

    lof = LocalOutlierFactor(
        contamination=contamination
    )

    labels = lof.fit_predict(values)

    anomalies = labels == -1

    plt.figure(figsize=(12, 5))

    plt.plot(
        temperature_series.index,
        temperature_series.values,
        label="Temperature"
    )

    plt.scatter(
        temperature_series.index[anomalies],
```

```
        temperature_series.values[anomalies],
        color="red",
        label="Anomalies"
    )

    plt.title(
        "LOF Analysis of Temperature"
    )

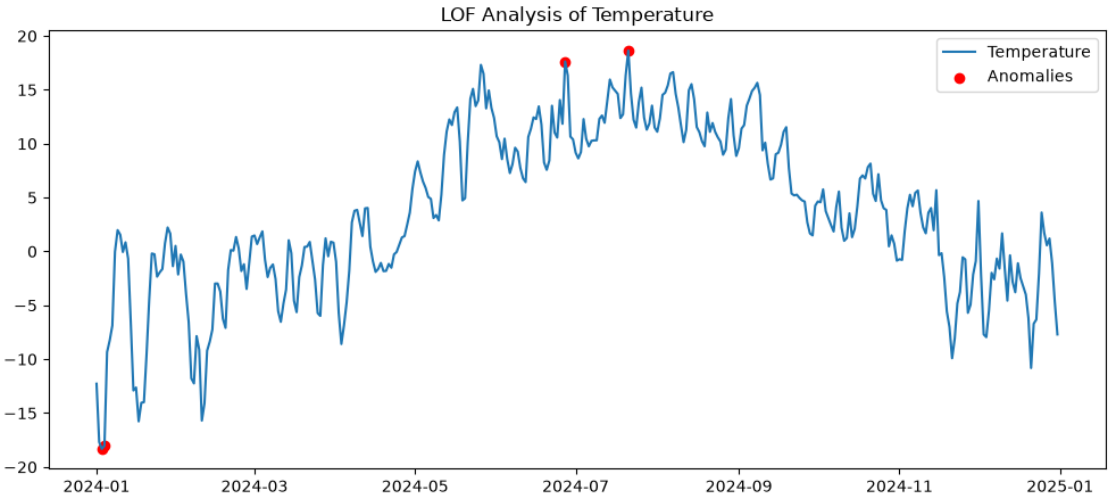
    plt.legend()

    plt.show()

    return {
        "n_anomalies": anomalies.sum(),
        "anomalies": anomalies
    }
```

[67]:
```
lof_results = temperature_lof(
    daily_weather["temperature_2m"],
    contamination=0.01
)

lof_results["n_anomalies"]
```



[67]:
```
np.int64(4)
```

[72]:
```
import requests
import pandas as pd
```

```
url = (
    "https://biapi.nve.no/magasinstatistikk/api/"
    "Magasinstatistikk/HentOffentligData"
)

response = requests.get(url)

magasin_data = response.json()

magasin_df = pd.DataFrame(
    magasin_data
)
```

[75]:
```
filling_df = (
    magasin_df[
        (magasin_df["omrType"] == "EL")
        &
        (magasin_df["omrnr"] == 1)
    ]
    .copy()
)

filling_df["dato_Id"] = pd.to_datetime(
    filling_df["dato_Id"]
)

filling_df = filling_df.sort_values(
    "dato_Id"
)

filling_series = (
    filling_df
    .set_index("dato_Id")
    ["fyllingsgrad"]
)
```

[76]:
```
filling_series.head()
```

[76]:
```
dato_Id
1995-01-08    0.617574
1995-01-15    0.579205
1995-01-22    0.546713
1995-01-29    0.504545
1995-02-05    0.465335
Name: fyllingsgrad, dtype: float64
```

[80]:
```
#STL Analysis
from statsmodels.tsa.seasonal import STL
```

[83]:
```
def stl_analysis(
    series,
    seasonal=53,
    trend=105,
    robust=True
):

    stl = STL(
        series,
        seasonal=seasonal,
        trend=trend,
        robust=robust
    )

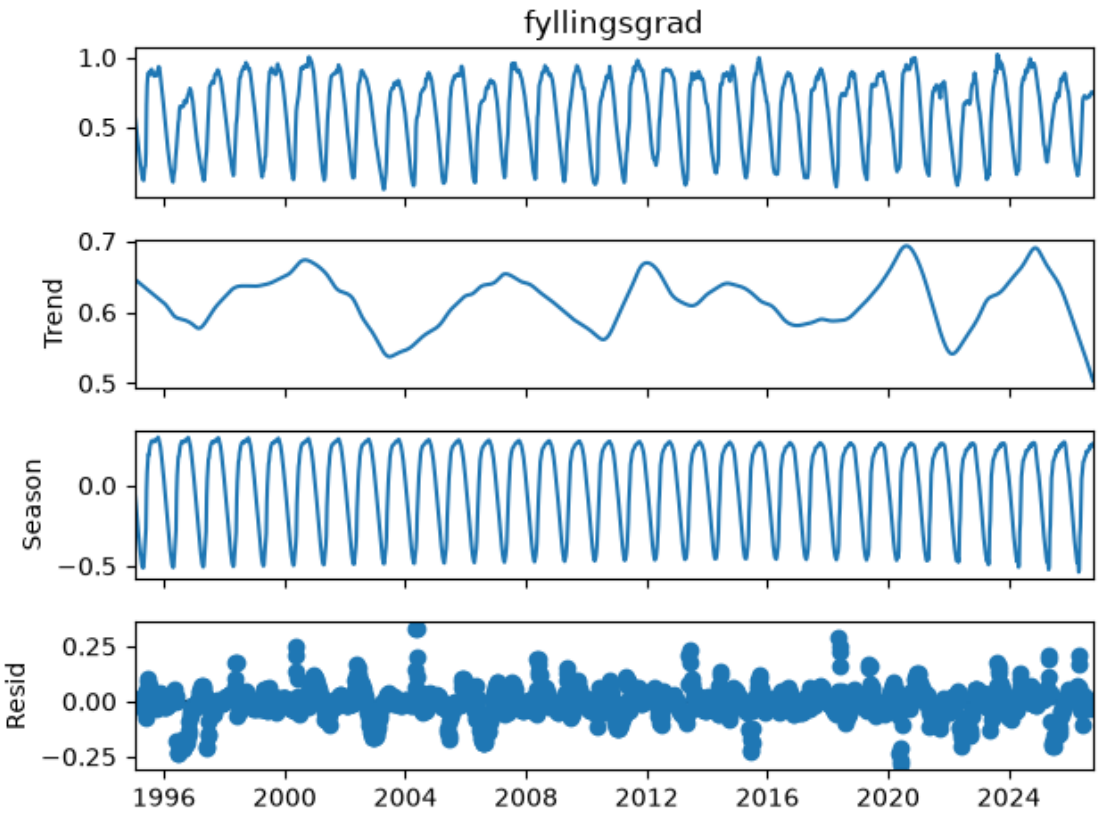
    result = stl.fit()

    fig = result.plot()

    plt.show()

    return result
```

[84]:
```
stl_result = stl_analysis(
    filling_series,
    seasonal=53,
    trend=105,
    robust=True
)
```



[85]:
```
from scipy.signal import spectrogram
```

[91]:
```
from scipy.signal import spectrogram
import numpy as np
import matplotlib.pyplot as plt


def spectrogram_analysis(
    series,
    window_length=128,
    overlap=64
):

    # Fjern gjennomsnittet
    series_centered = (
        series.values
        - np.mean(series.values)
    )

    frequencies, times, spectrum = spectrogram(
        series_centered,
```

```
        nperseg=window_length,
        noverlap=overlap
    )

    # Log-skala for bedre kontrast
    spectrum_db = 10 * np.log10(
        spectrum + 1e-12
    )

    plt.figure(figsize=(12, 5))

    plt.pcolormesh(
        times,
        frequencies,
        spectrum_db,
        shading="gouraud"
    )

    plt.ylabel("Frequency")
    plt.xlabel("Time")

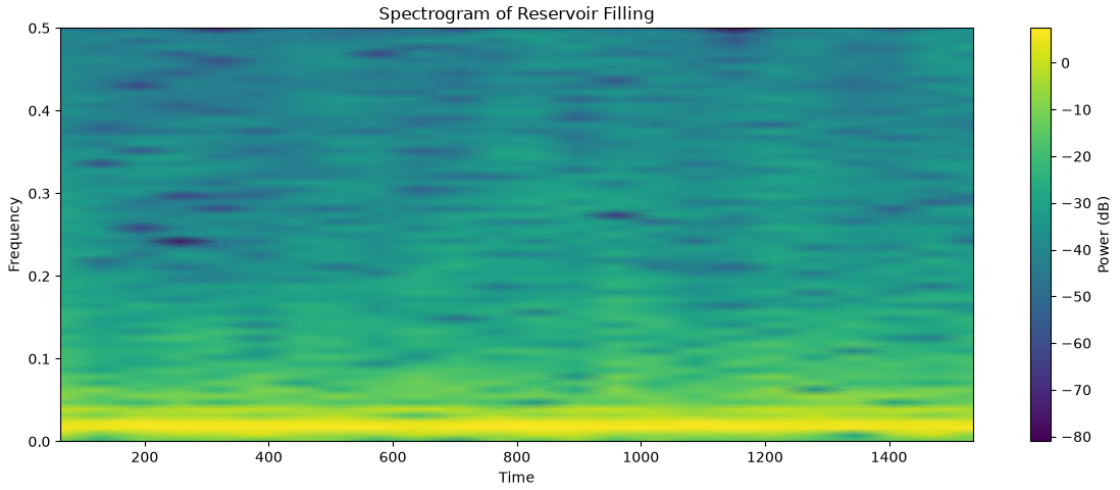
    plt.title(
        "Spectrogram of Reservoir Filling"
    )

    plt.colorbar(
        label="Power (dB)"
    )

    plt.tight_layout()
    plt.show()

    return {
        "frequencies": frequencies,
        "times": times,
        "spectrum": spectrum
    }
```

[92]:
```
spectrogram_result = spectrogram_analysis(
    filling_series,
    window_length=128,
    overlap=64
)
```



2.1 Log

The NVE regional division (RV, RM, RN, RØ and RS) was used as the geographical basis for the meteorological analysis. These regions correspond to the regional structure described in the assignment and were retrieved directly from the NVE Atlas ArcGIS service. To represent weather conditions within each region, random locations were sampled inside the regional polygons using a uniform random sampling procedure constrained by the polygon boundaries.

The sampled points were initially stored in the EPSG:25833 coordinate reference system and converted to WGS84 (EPSG:4326) before being submitted to the Open-Meteo API. Weather data from several sampled locations within Region Midt-Norge were compared before constructing regional datasets. Although local differences were observed, the locations followed similar seasonal patterns, supporting the use of regional aggregation.

Weather observations from multiple sampled locations were aggregated into a single representative regional time series by averaging temperature values and aggregating precipitation data. During development, a reduced sample size of 10 locations per region was used to limit API requests and testing time. The number of sampled locations was implemented as a configurable parameter to allow scaling if needed.

Hourly observations were aggregated to daily values to simplify visualization and improve the interpretability of the subsequent analyses. Daily precipitation was calculated as the sum of hourly precipitation, while daily temperature was calculated as the average of hourly temperatures.

For precipitation analysis, a Discrete Cosine Transform (DCT) was applied to remove low-frequency seasonal variation. The resulting seasonally adjusted series was analysed using a robust Statistical Process Control (SPC) approach. Control limits were calculated using the Median Absolute Deviation (MAD) instead of the standard deviation, reducing sensitivity to extreme events. Outliers were identified from the filtered series, while the original precipitation data were retained for visualization.

For temperature analysis, the Local Outlier Factor (LOF) method was used to identify anomalous observations. The contamination parameter was implemented as a configurable setting with a

default value of 1%, allowing the proportion of detected anomalies to be adjusted.

Weekly reservoir filling data from the NVE Magasinstatistikk API were analysed using Seasonal-Trend Decomposition using LOESS (STL). The seasonal and trend smoothing windows were selected to reflect annual filling cycles and longer-term developments in reservoir storage. In addition, a spectrogram was produced to examine the frequency characteristics of the reservoir filling series. The series was centred prior to spectral analysis to improve visibility of recurring patterns.

All major analyses were implemented as reusable functions with configurable parameters and tested directly in the notebook before integration into the Streamlit application.